Conceptual Thinking

Domain Modeling basics, Concepts, Multiplicity, Business rules, Generalisation

KdG Karel de Grote Hogeschool

Plan of action Class Diagram

Plan of action: Conceptual class model/Domain model

1. search for concepts
2. draw the classes
3. add attributes
4. add associations
5. validate

Plan of action: 1. search for concepts

step 1.1: start with nouns

Indicate all nouns

- These are candidates to be used as a concept
  - Inaccurate method, but good to start with
  - Filter out the good candidates
  - No hard and fast rule, but it gives you some guidelines
- Characteristics of good candidates
  - Is this data that we need to keep track of/follow up on?
  - Does the concept have one or more attributes?

Plan of action: 1. search for concepts

Characteristics of bad candidates

- Irrelevant, general or vague term
- Technical term outside the company domain (scanner, email, etc.)
- Synonym, plural, report... Choose one term as a concept, list others as synonyms, alternative representations
- Simple data type: e.g., just a number, text, date...
  - Does this describe another concept? List it as a candidate attribute for that concept.
- Leave out everything (everyone) that interacts with the system but is not part of it
  - the software system will usually not need to maintain that data (e.g. the logistics department is a user of a system)

Demo-1 : Car Dealer

Search for concepts using nouns

Mortsel Motors, a car dealership, wants to build a system to support the sales process. The dealer represents several brands. All available models are listed in a catalog. To support the sales process, the dealer has access to all kinds of information about the models and their manufacturers, such as the brand name, catalog value, purchase price, model name, body type, and chassis number. After the sale, the dealer stores information about the sale, such as the date of sale, the vehicle sold, and the sales price. The dealer also has contact information for its customers. Each customer has exactly one salesperson as their point of contact, ensuring personalized service.

Demo-1 : step 1.2 - filtering

noun	type	class name	remark
car dealership	concept	-	there is only 1? coat rack class!!
system	-	-	technical and general term
sales process	concept	sale	general term
dealer	synonym	-	
brands	concept / attribute	brand	
models	concept / attribute	model	
catalog	concept?	-	report can be generated
information		-	everything is information
vehicle	concept	car	
manufacturers	concept / attribute	manufacturer	
brand name	synonym	name	-> brand is a concept with a name

Demo-1 : step 1.2 - filtering

catalog value	attribute	cataloguePrice	
body type	attribute	carBodyStyle	
model name	attribute	name	-> model is a concept with a name
purchase price	attribute	purchasePrice	
chassis number	attribute	serialNumber	
sell	synonym	sale	
sale date	attribute	saleDate	
sales price	attribute	salePrice	
customer	concept	customer	
contact information	multiple attributes	name, address, id	
salesperson	concept	salesperson	not just a user of the system

Plan of action: 2. draw classes

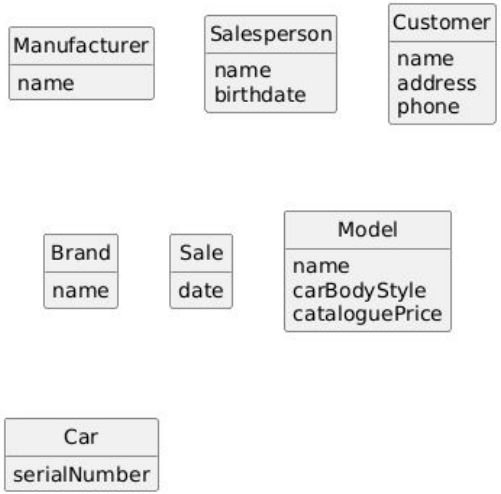
Draw only the necessary concepts in a conceptual class diagram (we also call this a domain model)

Draw each concept as a rectangle (i.e. a class)

Plan of action: 3. attributes

We also found some props!

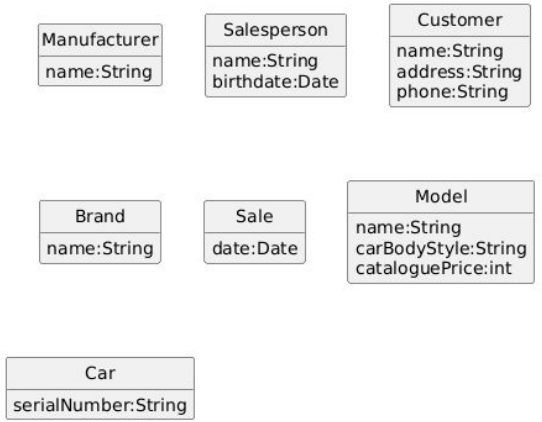
- Draw a line under the name of the class
- List all attributes below the line
  - if there isn't one, use name



optional: Indicates what data type each attribute requires (int, float, String, Date…)

- Data type comes after attribute, separated by a colon

  i.e name:String, birthdate:Date, …



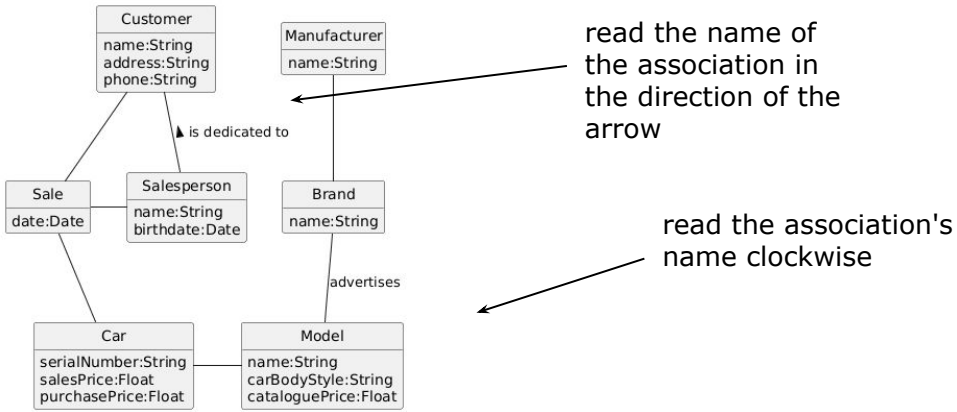
Plan of action: 4. Add associations

Some concepts are related

- e.g.: manufacturer manages multiple brands
- Add associations (relationships) between related concepts by drawing a line (NOT AN ARROW) between the classes.

Add a name to the relationship if it is unclear.

- Use clockwise notation or use an arrow in the name to indicate the reading direction. e.g., from > to e.g., Seller 'is dedicated to' Customer, Brand 'advertises' Model



Note: The arrow on the association name does not say anything about the relationship itself, only how to read it

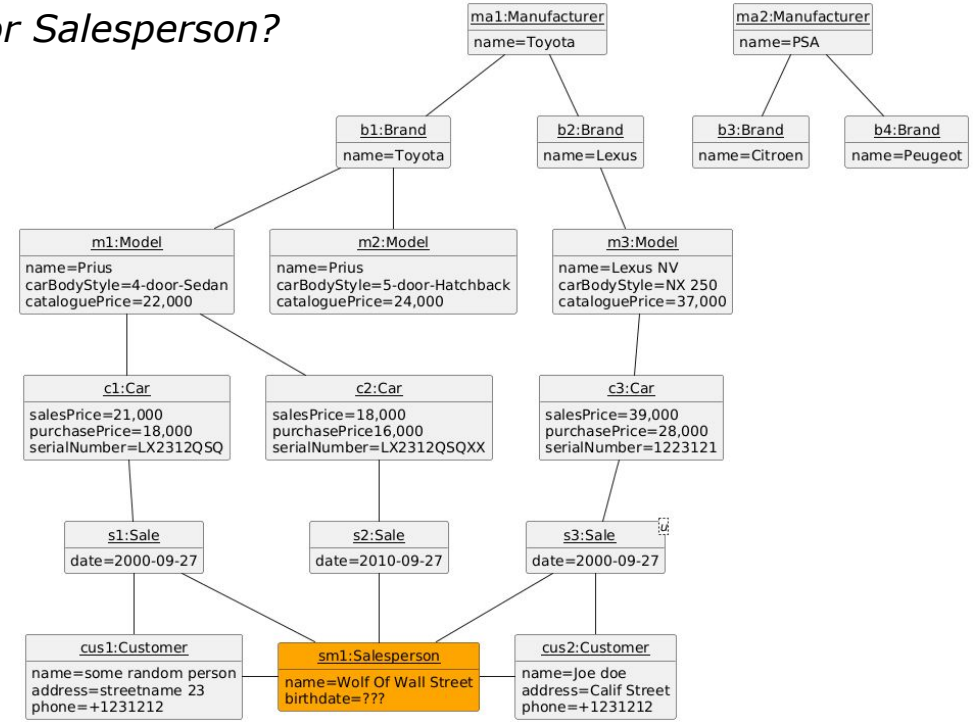
Action plan: 5. Validate using sample objects

For each class, think of 2 or 3 relevant examples

Link these examples together

Find errors in relationships or attributes…

e.g. Why do we keep track of dates for Salesperson?

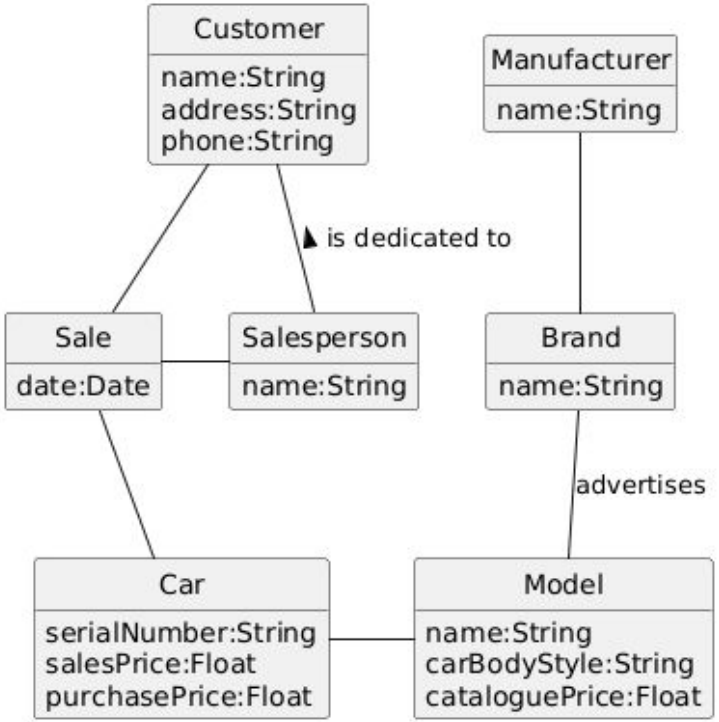


syntax for Object
unique_name:ClassName

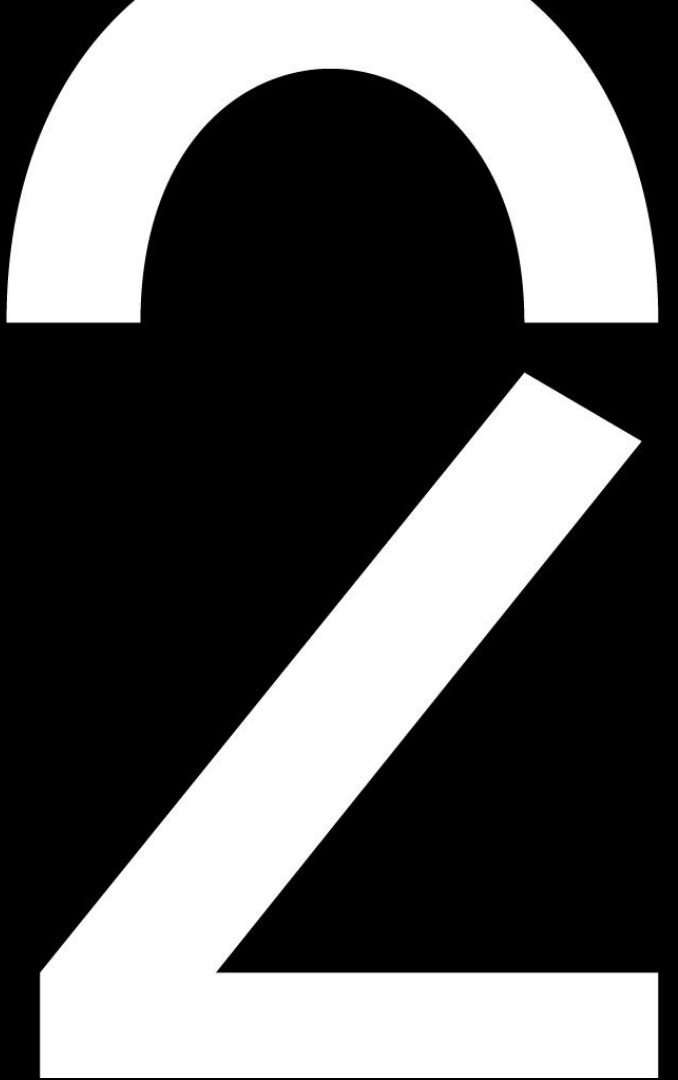
---

attribute=attributeValue

Demo-1 : Final Class Diagram



Alternative method: Find concepts using patterns



Find concepts by pattern: transaction

1. Are there transactions happening in the system?
   - Please note: Most projects aim to automate business transactions
   - Actions (verbs) are not concepts
   - But those actions can be transformed into concepts
     - e.g. flying somewhere → a flight / registering students → registration
     - Mortsel Motors → Sale

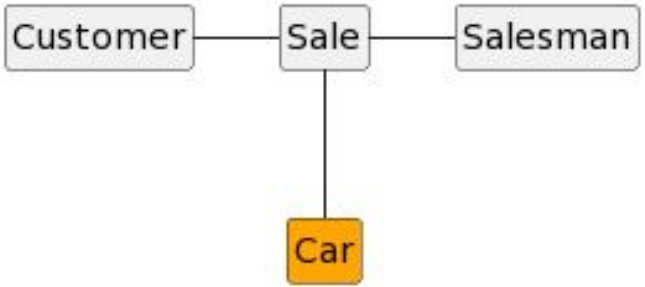
Domain concept pattern: consumer/supplier

2. In a transaction there is a consumer who receives a service from a supplier
   - AKA client/server, consumer/producer, request/response, supply/demand;
   - Both parties are therefore also related through the services provided
     - Mortsel Motors → Salesman/Customer

Domain concept pattern: product or service

3. In a transaction there is a supplier who provides a service to a customer: what is the service/product that is the subject of the transaction?

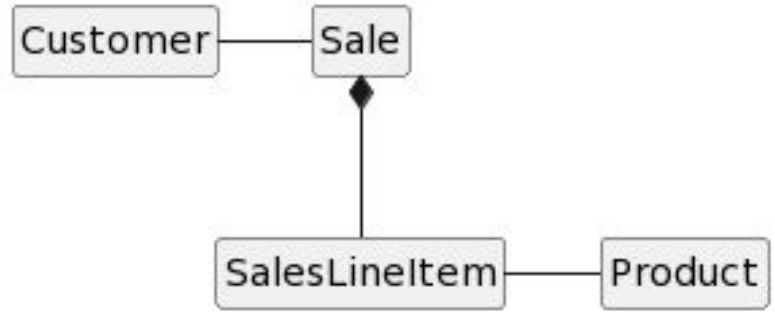
- Mortsel Motors: Car



Domain concept pattern: whole/part

4. Is any of the concepts made up of different parts? Is any of the concepts part of a larger whole?

- whole/part relationship, composition, collection, container
  - Mortsel Motors: not applicable
  - Alternative example: a supermarket sale consists of many items: 5 packs of chocolate, 3 jars of honey



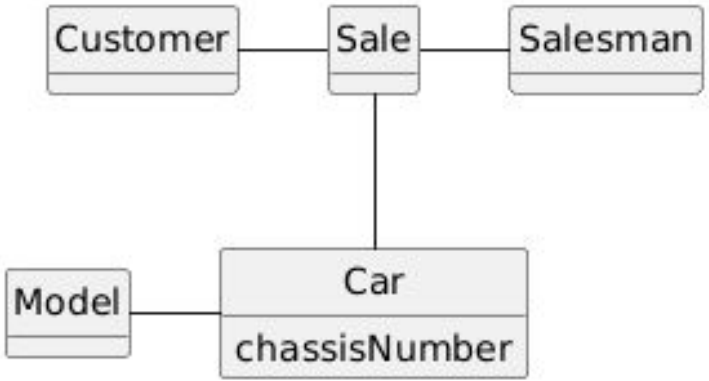
Additional information The black diamond indicates a UML composition: it is an exclusive whole/part relationship: a SalesLineItem belongs exclusively to a Sale and cannot be moved to another Sale

https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-aggregation-vs-composition/

Domain concept pattern: Descriptions

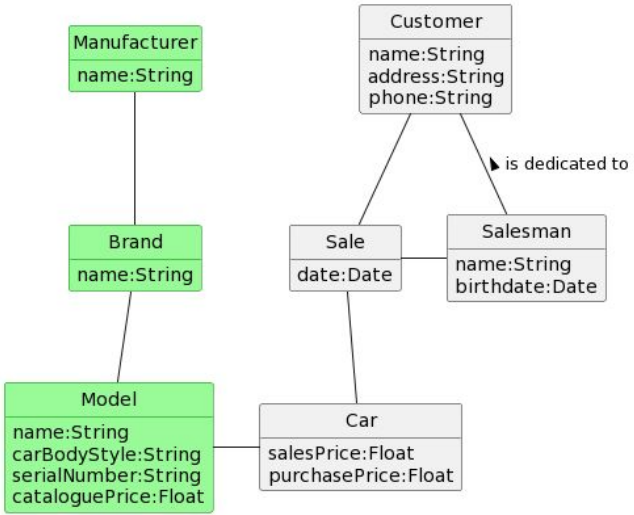
5. Is there a description shared by multiple elements? Do we need to maintain multiple copies of something?

- Mortsel Motors: We must keep track of the chassis number of every car sold. All cars share the characteristics of a Model
- In the supermarket example (previous slide) this was NOT necessary: The supermarket did not care what type of honey was sold.



Domain concept pattern: missing info

- You now have the core of your model. Now go find all the missing information.
  - Some info becomes simple attributes
  - Some info is more complex: draw them as associated concepts



Multiplicity
Business Rules
Generalization

Multiplicity

Multiplicity describes how many instances of one class can be connected to an instance of another class through a given association.

Examples of Multiplicity

1	Exactly one, no more and no less
0..1	Zero or one
*	Many
0..*	Zero or many
1..*	One or many
1..5	Between one and five

This is not an exhaustive list.

Multiplicity

Taking the example below, a customer has 1 or more bank accounts and a bank account can belong to one or two customers.

What about in the example below, what is the multiplicity?

Business Rules

Business rules define the constraints, conditions, and logic that govern how the system operates and how data in the system must behave. They ensure that the model reflects real-world policies and practices, not just technical relationships.

Examples of Business Rules

- A customer must have at least one bank account
- A bank account belongs to one or two customers
- A company must have at least one employee
- An employee works at only one company
- …

Demo: Train schedule for IC 2604

Task

In the brochure on the right we see the schedule of line IC-08, the IC 2604 route, both on weekdays (blue) and weekends and public holidays (yellow)

a. Sketch a domain model using a UML class diagram to capture the journey information for all intercity journeys on line IC-08. Write down all the business rules you can derive from your domain model and number them.

b. Don't forget multiplicity because from this week it is automatically part of the domain model.

c. Implement at least one generalization. You may add additional classes to the solution if it's not possible to draw a generalization for the existing classes.

d. extra: Draw an object diagram that includes all the data from the brochure

	IC 2604
Herkomst	
Antwerpen-Centraal	4.49
Antwerpen-Berchem ○	4.53
Antwerpen-Berchem	4.54
Mechelen ○	5.06
Mechelen	5.08
Brussels Airport - Zaventem ✈ ○	5.19
Brussels Airport - Zaventem ✈	5.21
Leuven ○	5.34
Leuven	5.42
Aarschot ○	5.53
Aarschot	5.55
Diest ○	6.07
Diest	6.09
Hasselt ○	6.22
Hasselt	
Diepenbeek	
Bilzen	
Tongeren ○	
Bestemming	

Line IC-08	IC 2604
Herkomst	
Antwerpen-Centraal	4.36
Antwerpen-Berchem ○	4.40
Antwerpen-Berchem	4.41
Mechelen ○	4.54
Mechelen	4.56
Brussels Airport - Zaventem ✈ ○	5.07
Brussels Airport - Zaventem ✈	5.09
Leuven ○	5.23
Leuven	
Wezemaal	
Aarschot ○	
Aarschot	
Langdorp	
Testelt	
Zichem	
Diest ○	
Diest	
Schulen	
Hasselt ○	
Bestemming	

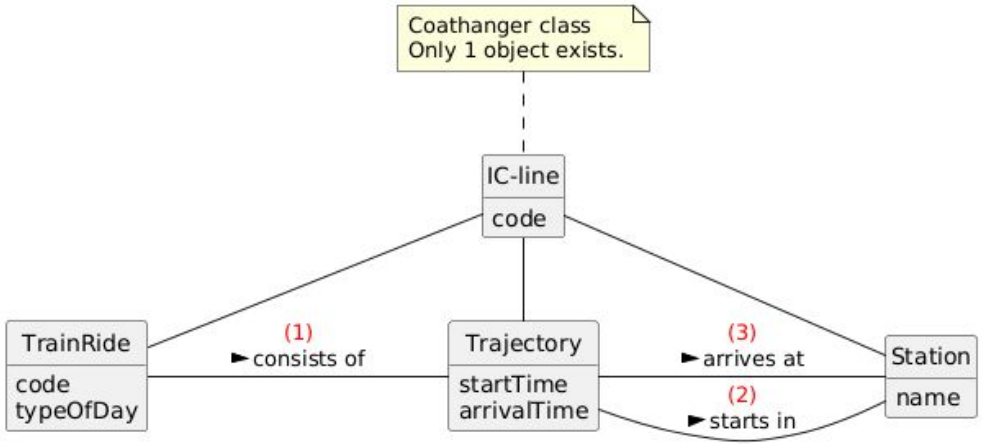
Note: This assignment is different from exercise 2.1. Line IC-08 is named, and the arrival and departure times of the train are also indicated. For simplicity's sake, we'll assume we're only including information about where this train actually runs. So not after Hasselt (L) or Leuven (R).

Demo: Train schedule for IC 2604

(a) Business rules

L→R
1a) Each ride consists of 1 or more legs
2a) Each route starts at 1 station
3a) Each route arrives at 1 station

R→L
1b) Each route corresponds to exactly 1 trip
2b) Each station can be the starting station for 0 or more routes
3b) Each station can be the end station for 0 or more routes

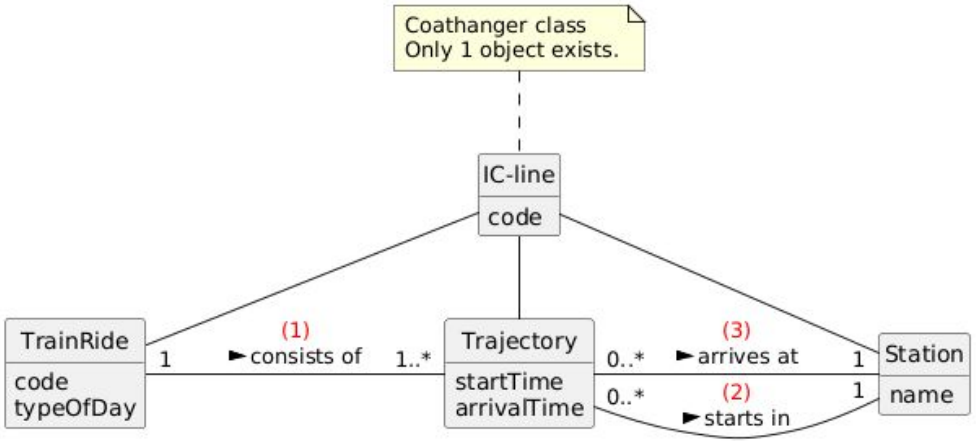


Demo: Train schedule for IC 2604

(b) Multiplicity

L→R
1a) Each ride consists of 1 or more legs
2a) Each route starts at 1 station
3a) Each route arrives at 1 station

R→L
1b) Each route corresponds to exactly 1 trip
2b) Each station can be the starting station for 0 or more routes
3b) Each station can be the end station for 0 or more routes



Plan of Action: generalizations

How to detect a correct generalization? A potential subclass must conform to:

100% Rule:

- All properties (attributes, associations and methods) of the superclass must be 100% applicable to the subclass.
- If a superclass has a property that the subclass does not have, then it is an incorrect generalization.

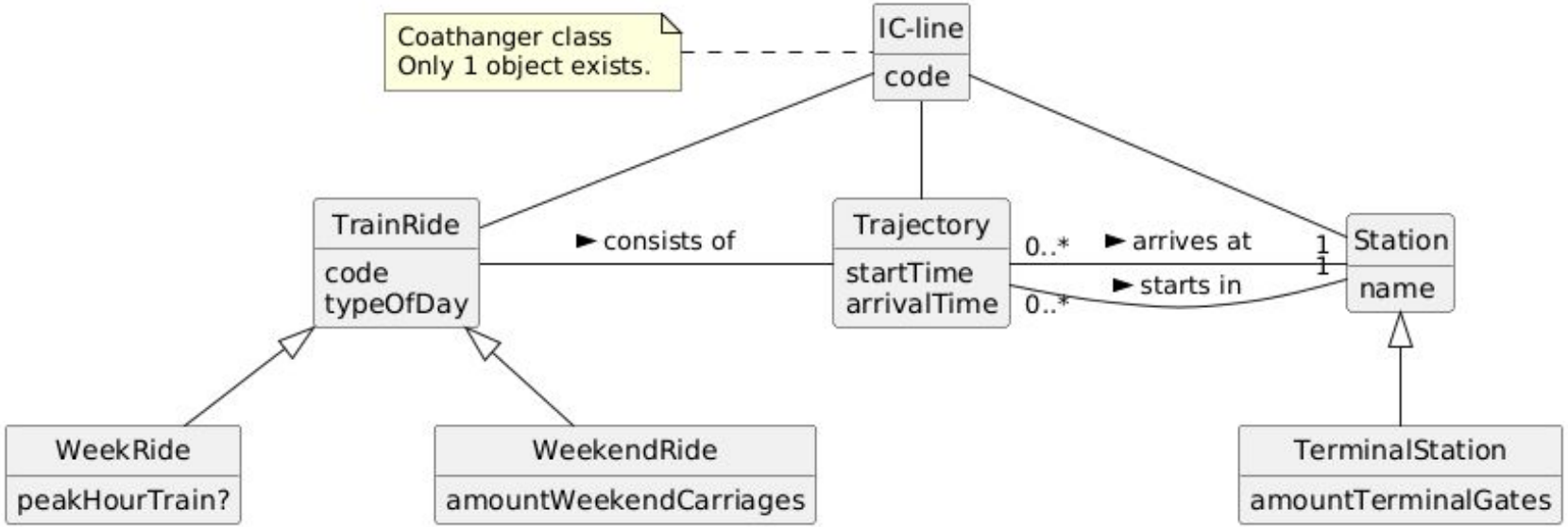
"Is-a" Rule:

- All members of a subclass collection must also be members of the superclass collection.
- Don't limit yourself to "is a", but use "Is every object of the subclass also an object of the superclass?"

  Example: not only this test: Max is an animal (this is a classification) but also this test: Is every object of the class dog also an object of the class animal?

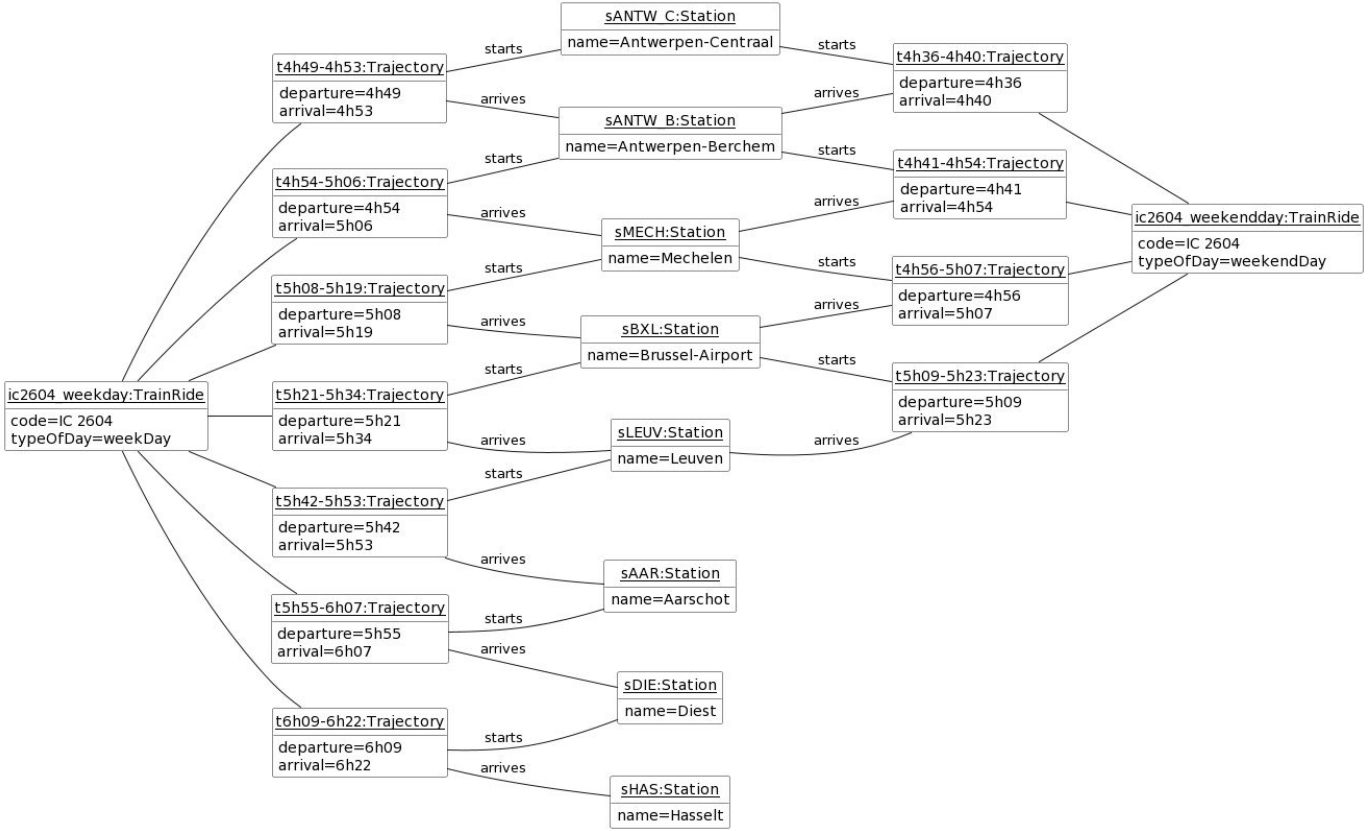
Demo: Train schedule for IC 2604

(c) possible generalizations



Demo: Train schedule for IC 2604

(d) object diagram



Search online

4

Other resources

Still unclear?
Search online : steps to identify concepts or classes in OO programming ..

- https://medium.com/xebia-engineering/a-real-world-introduction-to-finding-classes-in-object-oriented-programming-languages-612eae35b802
- http://users.csc.calpoly.edu/~jdalbey/308/Resources/ooDesignHowTo.html
- https://archive.eiffel.com/doc/manuals/technology/oosc/finding/page.html

Running example: Monopoly game

Recurring Example: Monopoly

Purpose of the Exercise

This exercise is an ongoing case study in Conceptual Thinking. Each week, we apply what we've learned to a digital version of Monopoly.

Why do we choose Monopoly?

- Everyone knows it, or almost everyone
- It contains rich structures: players, money, rules, actions, spaces, cards
- It provides an ideal context for modeling object-oriented systems
- It is tangible and recognizable, making abstract concepts concrete

Recurring Example: Monopoly

Learning objective

Students understand how theory is translated into a working conceptual model by:

- Working iteratively with one consistent case
- Apply different analysis methods (use cases, class diagrams, ...)
- Gain insight into the relationship between models and system behavior

Quick Refresher: How Does Monopoly Work Again?

- 2–6 players move their pawn around the board with a roll of 2 dice
- Players buy properties they land on
- Other players pay rent when they land on someone else's property
- There are special subjects such as Chance, Community Fund, Tax, Prison, etc.
- Players can take out mortgages, trade properties, or go bankrupt
- Goal: Become the richest player by bankrupting others

Conceptual Class Diagram